

TicketFlow REST Server

Server REST concorrente in C con backend SQLite

Alessia Smeralda Brunello (544964)

Università degli Studi di Messina
Dipartimento MIFT - Laurea in Informatica

Anno Accademico 2024/2025

Struttura della presentazione

- 1 Problem Statement
- 2 Stato dell'Arte
- 3 Metodologia ed Implementazione
- 4 Risultati
- 5 Conclusioni e Sviluppi futuri

Obiettivo del progetto

Realizzare un server REST concorrente in linguaggio C e usare SQLite o Redis come backend.

Requisiti richiesti

- Server REST in C.
- Gestione concorrente dei client.
- Backend Redis oppure SQLite.
- Operazioni CRUD su una risorsa.
- Risposte HTTP strutturate.

Risorsa applicativa scelta

- Sistema di ticketing: *TicketFlow*.
- Ogni ticket contiene titolo, descrizione, luogo, categoria, priorità e stato.
- I dati restano salvati nel database anche dopo la chiusura del server.

REST

Le risorse sono identificate da URI e manipolate tramite metodi HTTP come GET, POST, PUT e DELETE.

Concorrenza

Un server concorrente serve più client senza bloccare l'intero servizio su una sola connessione.

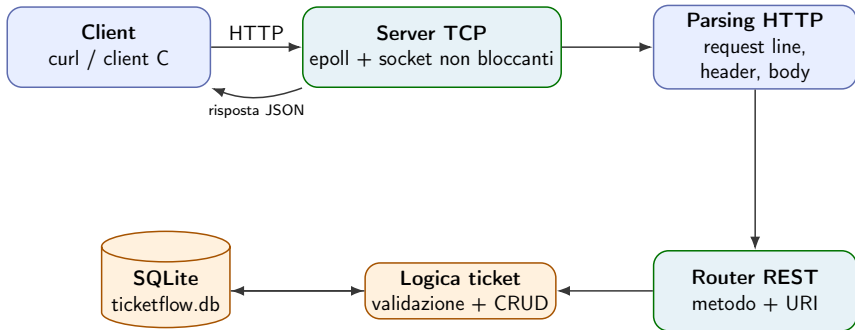
SQLite

SQLite offre un database leggero, integrato nel processo e adatto a progetti locali o prototipi backend.

Contesto del lavoro

Il progetto riproduce funzionalità tipiche di un piccolo backend web, ma implementate a basso livello: socket TCP, parsing HTTP, routing REST, accesso a SQLite e gestione degli errori.

Architettura generale



Il server è stato suddiviso in moduli indipendenti, ognuno con una responsabilità precisa, così da separare rete, HTTP, routing, logica applicativa e database.

- `server.c`: gestione della rete e delle connessioni;
- `http.c`: parsing delle richieste HTTP;
- `router.c`: associazione tra metodo, URI e operazione REST;
- `db.c`: accesso al database SQLite;
- `utils.c`: funzioni di supporto riutilizzabili.

API REST implementata

Metodo	Endpoint	Funzione	Esito
GET	/health	Verifica che il server sia attivo.	200
GET	/tickets	Restituisce l'elenco dei ticket.	200
GET	/tickets/{id}	Restituisce il dettaglio di un ticket.	200/404
POST	/tickets	Crea un nuovo ticket.	201
PUT	/tickets/{id}	Aggiorna un ticket esistente.	200/404
DELETE	/tickets/{id}	Elimina un ticket dal database.	200/404

Codici HTTP gestiti

- **200 OK** – richiesta completata correttamente.
- **201 Created** – ticket creato con successo.
- **400 Bad Request** – richiesta o contenuto non validi.
- **404 Not Found** – risorsa o ticket inesistente.
- **405 Method Not Allowed** – metodo non consentito per quella rotta.
- **413 Payload Too Large** – corpo della richiesta troppo grande.
- **500 Internal Server Error** – errore interno del server o del database.

Risultato

Il server gestisce esplicitamente i principali casi di errore e continua a servire le altre connessioni grazie all'architettura concorrente basata su `poll`.

Concorrenza e connessioni persistenti

Modello adottato

- Event loop basato su `epoll`.
- Socket in modalità non bloccante.
- Una sessione per ogni client connesso.
- Buffer per richieste parziali e risposte incomplete.

Aspetti HTTP

- Supporto a HTTP/1.1 con `keep-alive`.
- Controllo di `Content-Length`.
- Timeout per connessioni inattive.
- Risposte JSON con header HTTP completi.

Perché questa scelta

`epoll` evita un thread bloccante per ogni client e permette di gestire più connessioni in modo efficiente.

La versione HTTP/1.1 introduce le connessioni persistenti.

Tabella principale

- id: chiave primaria autoincrementale.
- title, description, location.
- category, priority, status.
- created_at: data di creazione.

Accesso ai dati

- Query preparate con SQLite.
- Funzioni dedicate per creazione, lettura, aggiornamento ed eliminazione.
- Mutex per serializzare l'accesso al database.
- Conversione dei risultati in JSON.

Risultato

SQLite rende persistenti i ticket: i dati non dipendono dalla memoria del processo e rimangono disponibili anche dopo il riavvio del server.

Risultati funzionali

Creazione ticket

POST /tickets

```
{'title':'Problema Wi-Fi', 'description':'La rete non funziona',  
'location':'Ufficio 2', 'category':'network',  
'priority':'high', 'status':'open'}
```

Risposta

HTTP/1.1 201 Created

Content-Type: application/json

Connection: keep-alive

```
{'message':'Ticket created', 'id':1}
```

Risultato

Il server valida il body JSON, salva il ticket su SQLite e restituisce una risposta HTTP in formato JSON.

Test di carico

Test	Req.	Conc.	Fallite	Req/sec	Tempo medio
KA Medio	500	20	0	9123.75	2.192 ms
KA Alto	1000	50	0	9632.52	5.191 ms
KA Stress	2000	100	0	9697.86	10.312 ms
Senza KA	1000	50	0	7727.92	6.470 ms

Interpretazione

I test principali sono stati eseguiti con keep-alive. Il confronto a parità di richieste e concorrenza mostra che il keep-alive aumenta il throughput e riduce il tempo medio di risposta.

PseudoCodice: event loop con epoll

Event loop del server:

- ❶ Attendi eventi da epoll
- ❷ Per ogni evento:
 - se è una nuova connessione → accetta il client
 - se il client invia i dati → leggi la richiesta
 - se la richiesta è completa → processala
 - se esiste una risposta → inviala
 - se il client chiude → libera la sessione
- ❸ Chiudi le sessioni inattive per timeout

Idea centrale

Il server usa `epoll` per controllare più sessioni client nello stesso ciclo, gestendo lettura, scrittura e timeout senza creare un thread per ogni connessione.

PseudoCodice: dal buffer HTTP al router

Parsing della richiesta HTTP:

- ❶ Leggi i dati ricevuti dal client (server.c)
 - ❷ Salvali nel buffer della sessione
 - ❸ Controlla se la richiesta è completa
 - ❹ Estrai: (http.c)
 - metodo HTTP, URI richiesta, header, eventuale body
 - ❺ Se la richiesta non è valida:
 - prepara una risposta di errore
- Altrimenti:
- passa la richiesta al router

Idea centrale

La richiesta viene ricostruita dal buffer della sessione, controllata tramite Content-Length, trasformata in `HttpRequest` e poi inviata al router REST.

PseudoCodice: accesso sicuro a SQLite

Accesso al database:

- 1 Acquisisci il mutex del database
- 2 Apri la connessione a SQLite
- 3 Prepara la query SQL con un prepared statement
- 4 Associa i valori del ticket ai placeholder della query
- 5 Esegui l'operazione sul database
- 6 Libera lo statement e chiudi la connessione
- 7 Rilascia il mutex

Idea centrale

Il mutex evita accessi concorrenti non controllati al database, mentre le query preparate separano il codice SQL dai dati ricevuti dal client.

In conclusione, *TicketFlow REST Server* rappresenta una buona base per comprendere concretamente come costruire un servizio REST partendo dai socket TCP.

I test funzionali e i test di carico hanno confermato il corretto funzionamento del flusso CRUD e la stabilità del server con più richieste concorrenti. In particolare, il supporto a keep-alive mostra il vantaggio di mantenere aperta la connessione quando vengono inviate più richieste dallo stesso client.

- Aggiungere l'autenticazione e l'autorizzazione sulle API per distinguere utenti diversi e limitare le operazioni.
- Introdurre HTTPS per proteggere la comunicazione client-server.
- Estendere il sistema con filtri, ricerca e ordinamento dei ticket.
- Valutare Redis per caching, code o gestione di dati temporanei.

Grazie per l'attenzione :)